

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability. (BL3, BL4)*

Activity Tool : Mock Interview (Low)

Assessment Tool Weightage: 30%

Faculty Name: Dr.G.Ayyappan

Student Reg.No:192424061

Name:K.Sandhya

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Interviewer: 'Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?'	BL3 – Apply	5
2	Interviewer: 'What is the difference between heuristic optimization and cost-based optimization? When would you use each?'	BL4 – Analyze	5
3	Interviewer: 'How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.'	BL3 – Apply	5
4	Interviewer: 'What are ACID properties? Give a real-world example where each property is critical.'	BL3 – Apply	5
5	Interviewer: 'Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?'	BL4 – Analyze	5
6	Interviewer: 'What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.'	BL4 – Analyze	5
7	Interviewer: 'Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?'	BL4 – Analyze	5

8	Interviewer: 'Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?'	BL3 – Apply	5
9	Interviewer: 'What is a serializability schedule? How do you test for conflict serializability using a precedence graph?'	BL4 – Analyze	5
10	Interviewer: 'Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?'	BL4 – Analyze	5

Code of Conduct:

I __K.Sandhya (192424061)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

K.Sandhya

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated

Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

1. Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?

Answer:

Query processing converts an SQL query into an efficient execution plan and produces the result.

Steps:

1. **Parsing and validation** – Checks SQL syntax, table names, column names, and data types.
2. **Query translation** – Converts SQL into an internal representation such as relational algebra.
3. **Query optimization** – Generates different possible execution plans.
4. **Cost estimation** – Estimates the cost of each plan using factors such as disk I/O, CPU time, memory, and number of tuples.
5. **Plan selection and execution** – Selects the lowest-cost plan and executes it.

Example: For joining Student and Department, the optimizer may compare Nested Loop Join, Hash Join, and Sort-Merge Join and choose the cheapest one.

2. Difference between heuristic optimization and cost-based optimization.

When would you use each?

Heuristic Optimization	Cost-Based Optimization
Uses predefined rules	Uses estimated execution costs
Does not calculate exact cost	Calculates cost for different plans
Faster optimization	More optimization time
Simple to implement	More complex
Example: perform selection before join	Example: choose cheapest join algorithm

When to use:

- **Heuristic:** When the query is simple and quick optimization is required.

- **Cost-based:** For complex queries where performance is important.

In practice, DBMSs commonly combine both approaches.

3. How would you implement database connectivity in a real project?

Explain JDBC and ODBC.

Answer:

In a real project, database connectivity can be implemented as follows:

1. Install the required database driver.
2. Create a database connection using the database URL, username, and password.
3. Create a connection object.
4. Use PreparedStatement to execute parameterized SQL queries.
5. Process the returned ResultSet.
6. Close the connection and other resources.

JDBC – Java Database Connectivity

- Used mainly for connecting Java applications to databases.
- Uses Java-specific APIs and database drivers.
- Example: Java application → JDBC → MySQL.

ODBC – Open Database Connectivity

- A standard interface that allows applications to connect to different databases.
- It is language-independent.
- Uses ODBC drivers.

Main difference: JDBC is specifically designed for Java, while ODBC is a more general database connectivity standard.

4. What are ACID properties? Give a real-world example.

ACID properties ensure reliable database transactions.

A – Atomicity

- A transaction is completed completely or not at all.
- **Example:** In a bank transfer, money must be deducted from one account and added to another. If one operation fails, the whole transaction is rolled back.

C – Consistency

- The database must remain valid before and after a transaction.
- **Example:** A bank balance cannot violate predefined rules.

I – Isolation

- Concurrent transactions should not interfere incorrectly with each other.
- **Example:** Two people booking the last available seat should not both receive the same seat.

D – Durability

- Once a transaction is committed, its changes remain permanently.
- **Example:** After a successful bank transfer, the transaction should remain even if the server crashes.

Easy memory: A-C-I-D = Atomicity, Consistency, Isolation, Durability.

5. Explain dirty read, lost update, and unrepeatable read. How does 2PL solve them?

1. Dirty Read

A transaction reads data written by another transaction before it is committed.

Example:

- T1 updates salary from ₹30,000 to ₹40,000.
- T2 reads ₹40,000.
- T1 rolls back.

- T2 has read invalid data.

2. Lost Update

Two transactions update the same data, and one update overwrites the other.

Example:

- T1 reads balance = ₹10,000.
- T2 reads balance = ₹10,000.
- T1 updates it to ₹9,000.
- T2 updates it to ₹8,000.
- T1's update is lost.

3. Unrepeatable Read

A transaction reads the same row twice and gets different values because another transaction modified it between the reads.

How 2PL helps:

Two-Phase Locking (2PL) has:

- **Growing phase:** Transaction obtains locks.
- **Shrinking phase:** Transaction releases locks.

Locks prevent conflicting transactions from accessing data incorrectly at the same time, helping maintain serializability and consistency.

6. What is deadlock in DBMS? Detection algorithm and prevention strategies.

Deadlock occurs when two or more transactions wait for each other indefinitely.

Example:

- T1 locks A and waits for B.
- T2 locks B and waits for A.

Neither transaction can continue.

Deadlock Detection – Wait-For Graph:

1. Create a node for each transaction.
2. Draw an edge $T1 \rightarrow T2$ if $T1$ is waiting for a lock held by $T2$.
3. Check the graph for a cycle.
4. If a cycle exists, a deadlock is present.
5. Select a transaction as a victim and rollback it to break the cycle.

Prevention strategies:

1. **Wait-Die:** Older transactions can wait; younger transactions are rolled back.
2. **Wound-Wait:** Older transactions can force younger transactions to roll back.
3. **Resource ordering:** Transactions request resources in a fixed order.

7. Compare Immediate Update and Deferred Update recovery.

Immediate Update	Deferred Update
Database may be updated before commit	Database is updated only after commit
Requires UNDO for uncommitted changes	Usually requires REDO
Recovery is more complex	Recovery is simpler
Log records are important	Log records are also maintained
Useful when early database updates are needed	Useful when simpler recovery is preferred

Immediate Update Example:

A transaction changes a customer's balance before it commits. If it fails, the old value must be restored.

Deferred Update Example:

Changes are kept in the log/buffer and applied to the database only after successful commit.

Preferred use:

- Immediate update $\rightarrow$ systems where updates can occur before commit.

- Deferred update → systems where simpler recovery and avoiding uncommitted database changes are preferred.

8. Explain Shadow Paging. Advantages and limitations compared to log-based recovery.

Shadow Paging is a recovery technique that maintains two page tables:

- **Shadow page table** – points to the original database pages.
- **Current page table** – points to modified pages.

When a transaction modifies a page, a new copy is created instead of changing the original page.

After successful commit, the current page table becomes the new database reference.

Advantages:

1. No complex UNDO/REDO log is required.
2. Recovery is fast.
3. Simple rollback.
4. Original pages remain unchanged until commit.

Limitations:

1. Page-table management can require extra space.
2. Can cause fragmentation.
3. Copying pages can be expensive.
4. Not ideal for very large databases or high-concurrency systems.

Compared with log-based recovery: Log-based recovery is generally more flexible and widely used in modern DBMSs, while shadow paging is simpler but can have higher space-management overhead.

9. What is a serializability schedule? How do you test conflict serializability?

A serial schedule executes transactions one after another without interleaving.

A serializable schedule may interleave transactions but produces the same result as some serial execution.

Conflict Serializability Test

Use a precedence graph.

1. Create one node for each transaction.
2. Check conflicting operations:
 - Read–Write
 - Write–Read
 - Write–Write
3. If transaction T1 performs the conflicting operation before T2, draw:
 $T1 \rightarrow T2$
4. Check the graph for a cycle.

Result:

- No cycle $\rightarrow$ Conflict serializable
- Cycle $\rightarrow$ Not conflict serializable

Example:

If the graph is:

$T1 \rightarrow T2 \rightarrow T3$

There is no cycle, so the schedule is conflict serializable.

If:

$T1 \rightarrow T2 \rightarrow T1$

There is a cycle, so it is not conflict serializable.

10. Explain Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used?

Two-Phase Locking (2PL) controls concurrent transactions using locks.

Type	Description
Basic 2PL	Has growing and shrinking phases. Locks can be released before commit.
Strict 2PL	All exclusive/write locks are held until commit or rollback.
Rigorous 2PL	All locks, both shared and exclusive, are held until commit or rollback.

Phases of Basic 2PL

Growing phase: Acquire locks, no locks are released.

Shrinking phase: Release locks, no new locks can be acquired.

Which is commonly used?

Strict 2PL is commonly used because it:

- Ensures conflict serializability.
- Prevents cascading rollbacks.
- Makes recovery easier.
- Provides strong isolation.

Easy memory:

Basic → release anytime after shrinking starts

Strict → write locks until commit

Rigorous → all locks until commit